



第 1 集 · 约 14 分钟

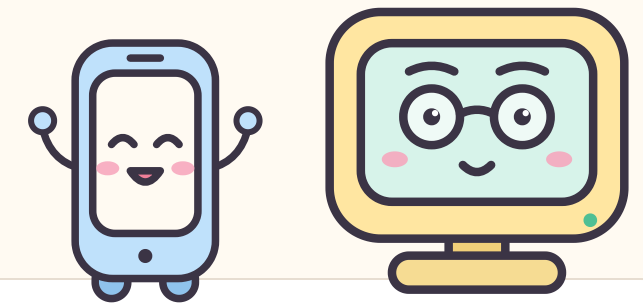
开课：我们要写一个什么样的程序

先看成品，再看图纸，最后备好工具

配套阅读：书的第 0 章



从零写出 LanShare



这一集你会学到

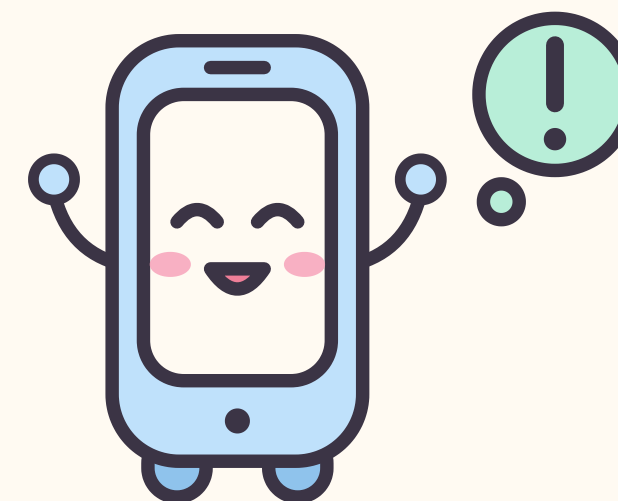
1. LanShare 能做什么，难点在哪里
2. 整体架构：手机浏览器、网络、电脑上的程序各做什么
3. 一个文件从手机到电脑经过的每一步
4. 这门课的路线图，和要装的工具



一个日常的小麻烦

小手机拍的视频，怎么最快地传给旁边的老电脑？

数据线、网盘、聊天软件都行，但要么得找线，要么绕一圈外网，要么被压缩画质。我们写一个只用浏览器、走局域网、全程加密的工具。





电脑上的主界面



电脑上的主界面（截图里的地址和口令是测试时的值）



手机上的页面



○ 选文件发给电脑

○ 电脑上的文件，点一下下载

○ 收到的文字，一键复制

手机浏览器里打开的页面（测试时的截图）



看起来简单，难在哪里

● 加密要在浏览器里做

局域网地址是 http，不是 https，浏览器不给网页用它自带的加密接口

● 两边的加密代码必须一模一样

我们用 Rust 写一份，编译两次：一次给电脑，一次编译成 WebAssembly 给浏览器

● 密钥怎么交给小手机，还不能被同一个 WiFi 里的窃听者截获

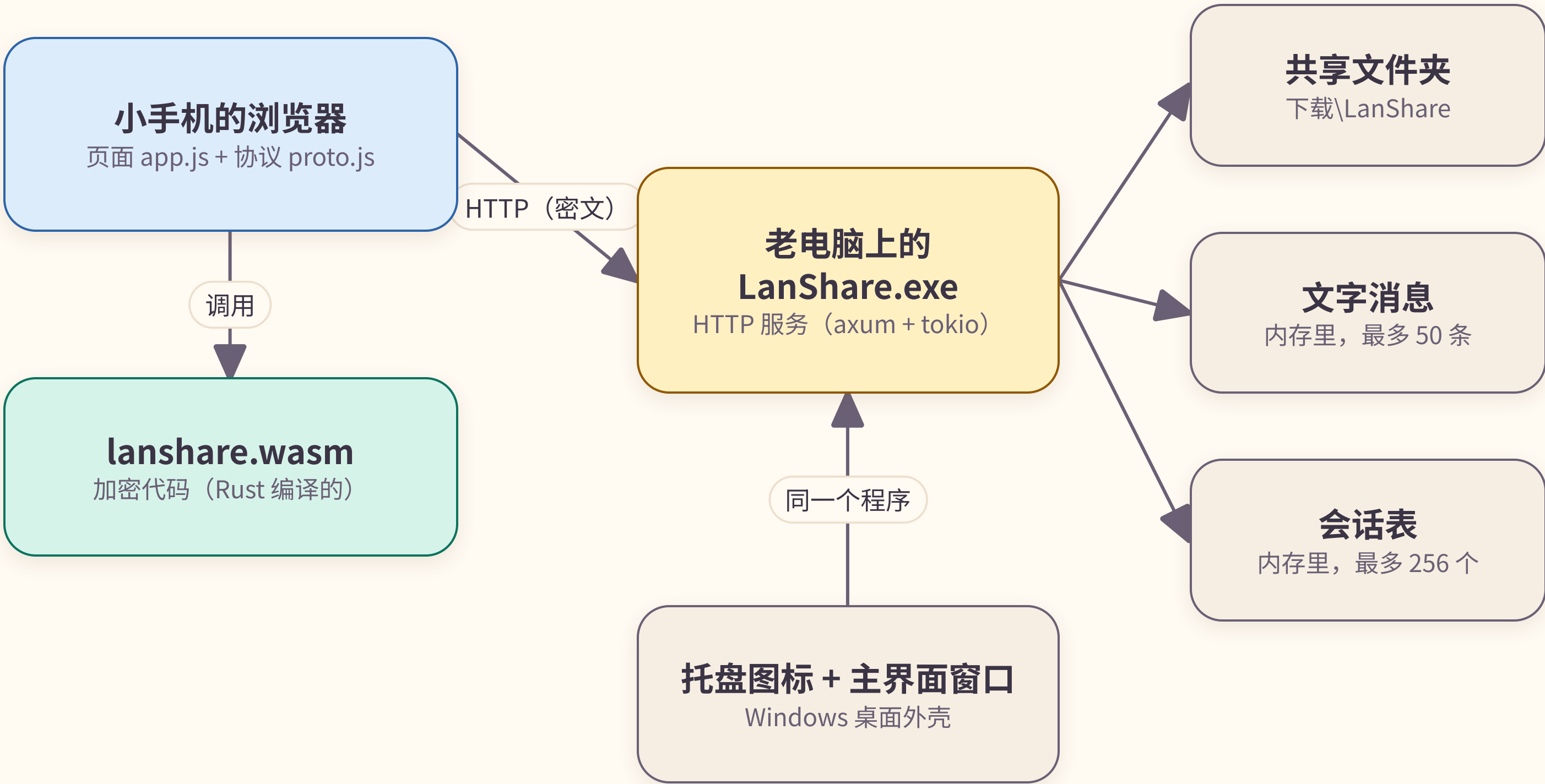
扫码时，密钥放在网址井号后面的部分，浏览器不会把它发到网络上

● 局域网里也有捣蛋鬼

猜口令、重放旧数据、占满连接、恶意文件名……三轮安全审查，每一条都有测试



整体架构





一个文件从手机到电脑



发生了什么

K 留在小手机里，不会发出去

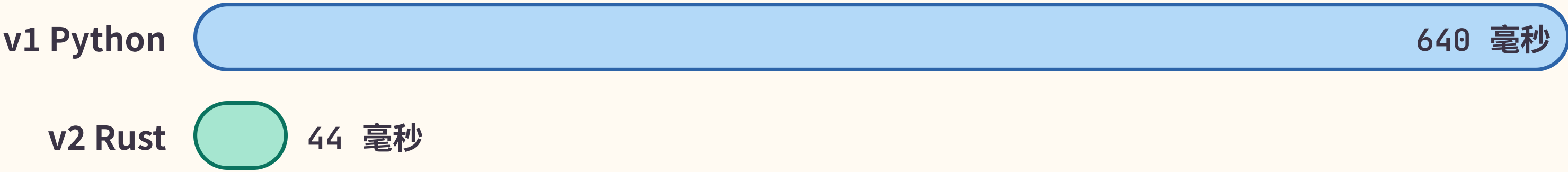
公开内容，谁都能拿

窃听器看得到，但算不出密钥

窃听器只看到乱码



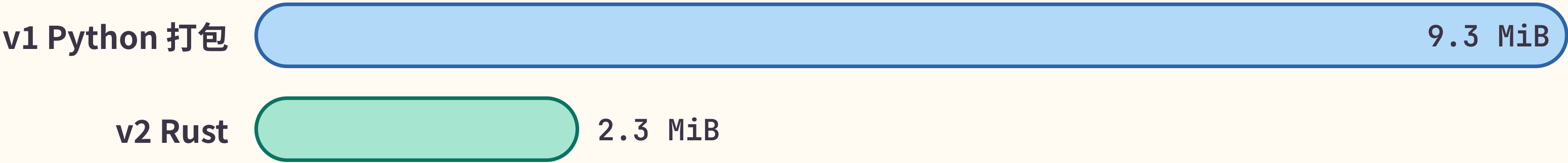
v1 和 v2：启动时间



冷启动到页面可访问，5 次取中位数；开发机 i9-12900H（数据见 docs/bench.md）



v1 和 v2：程序体积



单个 exe 文件的大小（数据见 docs/bench.md）



这门课的路线图

集	内容	写出来的东西
第 2 集	搭工作区骨架	三个 crate 的空架子
第 3~5 集	协议核心	加密信道、握手、口令登录、防重放、文件名清理
第 6~7 集	WebAssembly	把加密核心交给浏览器里的 JavaScript
第 8~15 集	服务端	命令行、存储、路由、连接管理、配对、加密接口、分块传输、测试
第 16~17 集	桌面外壳	程序入口、单实例、主界面窗口、托盘
第 18~19 集	网页	浏览器端的协议客户端和界面
第 20 集	打包和安装	一键构建、安装到开始菜单、卸载，最后复盘



我们要写多少代码

部分	行数（含注释和空行）
协议核心 lanshare-proto	347
WASM 导出层 lanshare-wasm	273
服务端 server 目录	1534
命令行、存储、日志、程序入口等	995
桌面外壳 desktop 目录	382
网页（HTML、CSS、JavaScript）	1149
测试（Rust 和 Node）	1921
打包、安装、卸载脚本	286



怎么学这门课



- 跟着敲，不要只看
- 每集结束，编译必须通过
- 编译报错先读一遍
- 对照完整源码



装工具



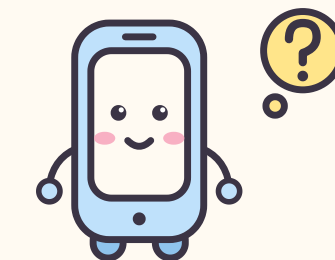
Windows PowerShell

```
PS D:\code> rustup --version
rustup 1.29.0 (28d1352db 2026-03-05)
info: This is the version for the rustup toolchain manager, not the rustc compiler.
info: the currently active `rustc` version is `rustc 1.96.0 (ac68faa20 2026-05-25)`

PS D:\code> rustup target add wasm32-unknown-unknown
info: component rust-std for target wasm32-unknown-unknown is up to date

PS D:\code> node --version
v24.12.0

PS D:\code> git --version
git version 2.45.2.windows.1
```



选择题

手机通过 http 开头的局域网地址打开页面，浏览器不给页面用它自带的加密接口。我们是怎么解决的？

- A 改用 https，给电脑申请一个证书
- B 把 Rust 写的加密代码编译成 WebAssembly，放进页面里
- C 不加密，局域网里本来就安全



选择题

手机通过 http 开头的局域网地址打开页面，浏览器不给页面用它自带的加密接口。我们是怎么解决的？

A 改用 https，给电脑申请一个证书

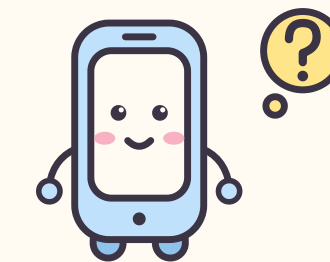
局域网 IP 申请不到正规证书；自签名证书会让浏览器弹出吓人的警告，也挡不住主动攻击。

C 不加密，局域网里本来就安全

同一个 WiFi 下的窃听者可以抓包，明文传输的所有内容都能被看到。

B 把 Rust 写的加密代码编译成 WebAssembly，放进页面里

浏览器能运行 WebAssembly，不管页面是不是 https。而且这样手机和电脑跑的是同一份加密代码。



选择题

二维码的网址里，密钥写在井号后面。这样做最大的好处是什么？

- A 网址更短，二维码更好扫
- B 井号后面的部分不会出现在网络请求里，所以密钥的明文不经过网络
- C 服务器可以更快地读到密钥



选择题

二维码的网址里，密钥写在井号后面。这样做最大的好处是什么？

A 网址更短，二维码更好扫

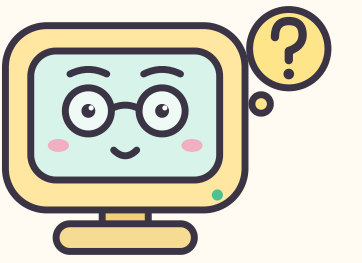
放在哪里长度都差不多，这不是原因。

B 井号后面的部分不会出现在网络请求里，所以密钥的明文不经过网络

按浏览器的规则，网址里井号后面的片段只留在浏览器里，不会发给服务器，同一个 WiFi 下的窃听者也看不到它。

C 服务器可以更快地读到密钥

恰恰相反，服务器根本收不到这一部分。服务器自己本来就知道 K，因为二维码就是它生成的。



这个工具只在家里或办公室的局域网里用，为什么还值得花这么大力气做端到端加密？



想一想

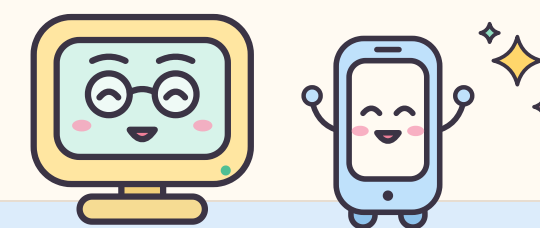
这个工具只在家里或办公室的局域网里用，为什么还值得花这么大力气做端到端加密？

同一个 WiFi 下的任何一台设备，都可能看到经过的网络流量：咖啡馆、酒店、公司的共享网络里，你不知道旁边连着谁。明文传输时，文件内容、文件名、文字消息都会被看光。加密之后，窃听者最多知道你传了多大的东西、什么时候传的。



这一集我们看了

- ✓ 成品：电脑上一个窗口加托盘，手机扫码用浏览器传文件和文字
- ✓ 难点：浏览器里加密、两端代码一致、密钥不经过网络、局域网里的各种攻击
- ✓ 架构：手机浏览器（app.js、proto.js、lanshare.wasm）和电脑上的 LanShare.exe
- ✓ 路线：协议 → WebAssembly → 服务端 → 桌面外壳 → 网页 → 打包安装
- ✓ 工具：Rust、wasm32 编译目标、Node.js、git



下一集

全课完结，感谢收看